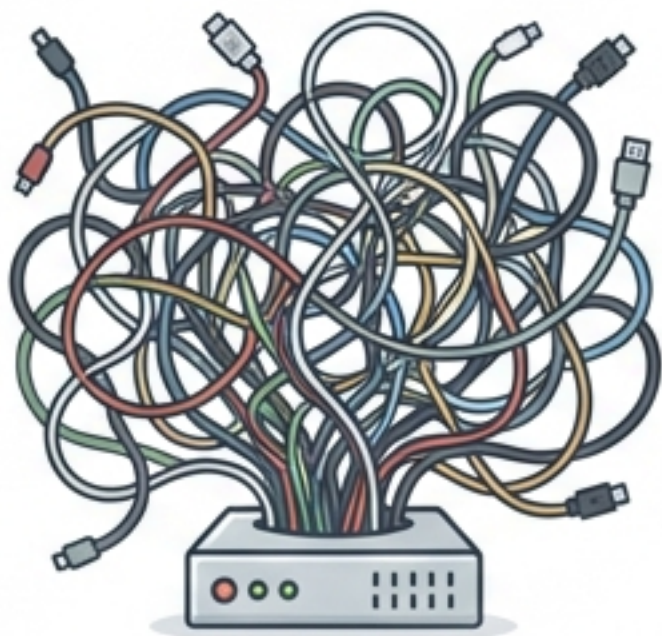




Aula 4: Respostas com HTML, Template Engines e EJS

Prof. Ronaldo Borges

O Problema



```
app.get('/', (req, res) => {  
  res.send('<h1>Olá ' + nome + '</h1><p>Bem-vindo  
    ao sistema. Seu saldo é ' + saldo + '</p>');  
});
```

Gargalo de Manutenção: Enviar strings de HTML cru via `res.send()` das strings HTML cru via `res.send()` mistura a lógica do servidor (Back-end) com a apresentação (Front-end). É insustentável para páginas complexas.

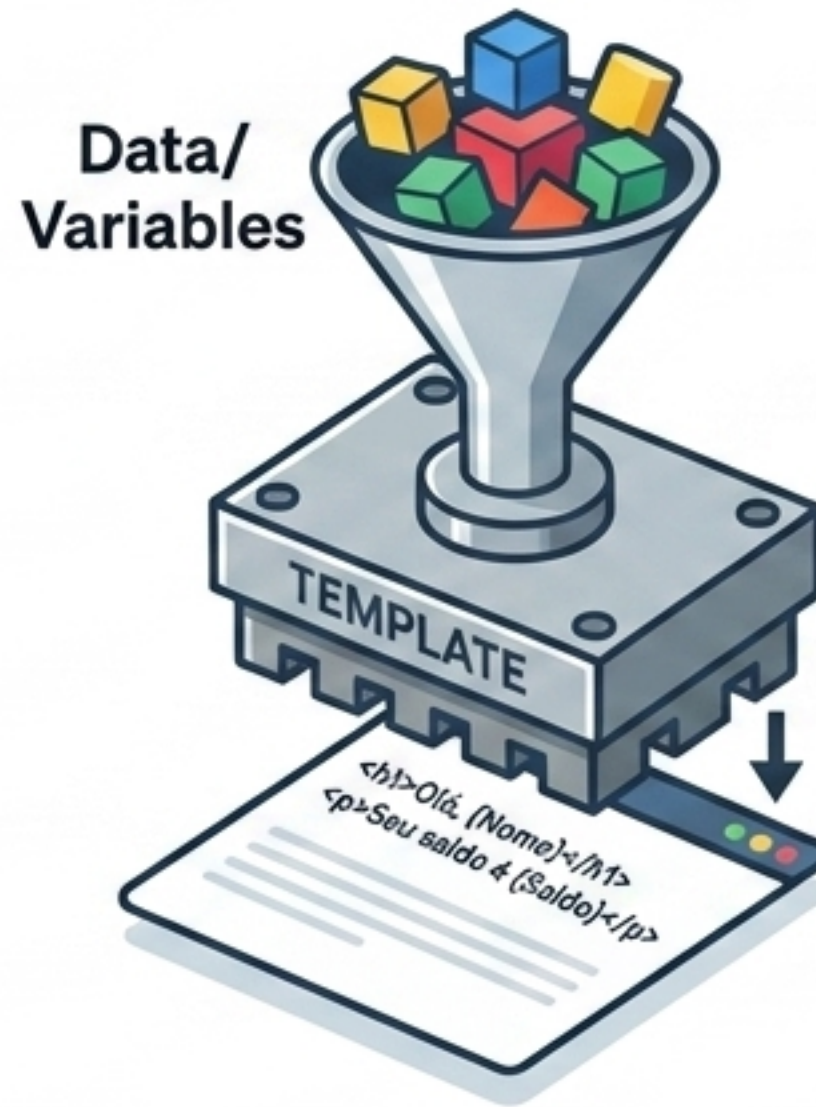
A Solução



A Separação Necessária: Precisamos de uma ponte estruturada. Um mecanismo que permita escrever HTML puro e injetar dados dinâmicos de forma elegante e organizada.

O que são Template Engines?

Definição: "Motor de modelos". Um mecanismo que usa templates pré-definidos para gerar conteúdo HTML final de forma dinâmica.



Função: Permite a incorporação de variáveis e lógica de programação (loops, condicionais) diretamente na estrutura do HTML.

Mustache

Handlebars

doT

EJS

PUG
(antigo Jade)

Conhecendo o EJS

<%= EJS %>

Embedded JavaScript templating.

O que significa o 'E'?

- Embedded (Embutido)? Sim, JavaScript embutido no HTML.
- Poderia ser também:
 - Eficaz
 - Elegante
 - Easy (Fácil)

A Filosofia

- É uma linguagem de modelagem simples.
- Permite gerar marcação HTML utilizando JavaScript simples (sem necessidade de aprender uma nova sintaxe complexa).
- Se você sabe JavaScript e HTML, você já sabe EJS.

○ EJS na Arquitetura MVC



O EJS separa completamente a lógica de rotas (Express) da interface do usuário (HTML).

Configuração no Express: Passo a Passo

1. Instalação via Terminal

```
$ npm init  
$ npm install --save express  
$ npm install --save ejs  
$ npm install --save ejs
```

2. Configuração no index.js

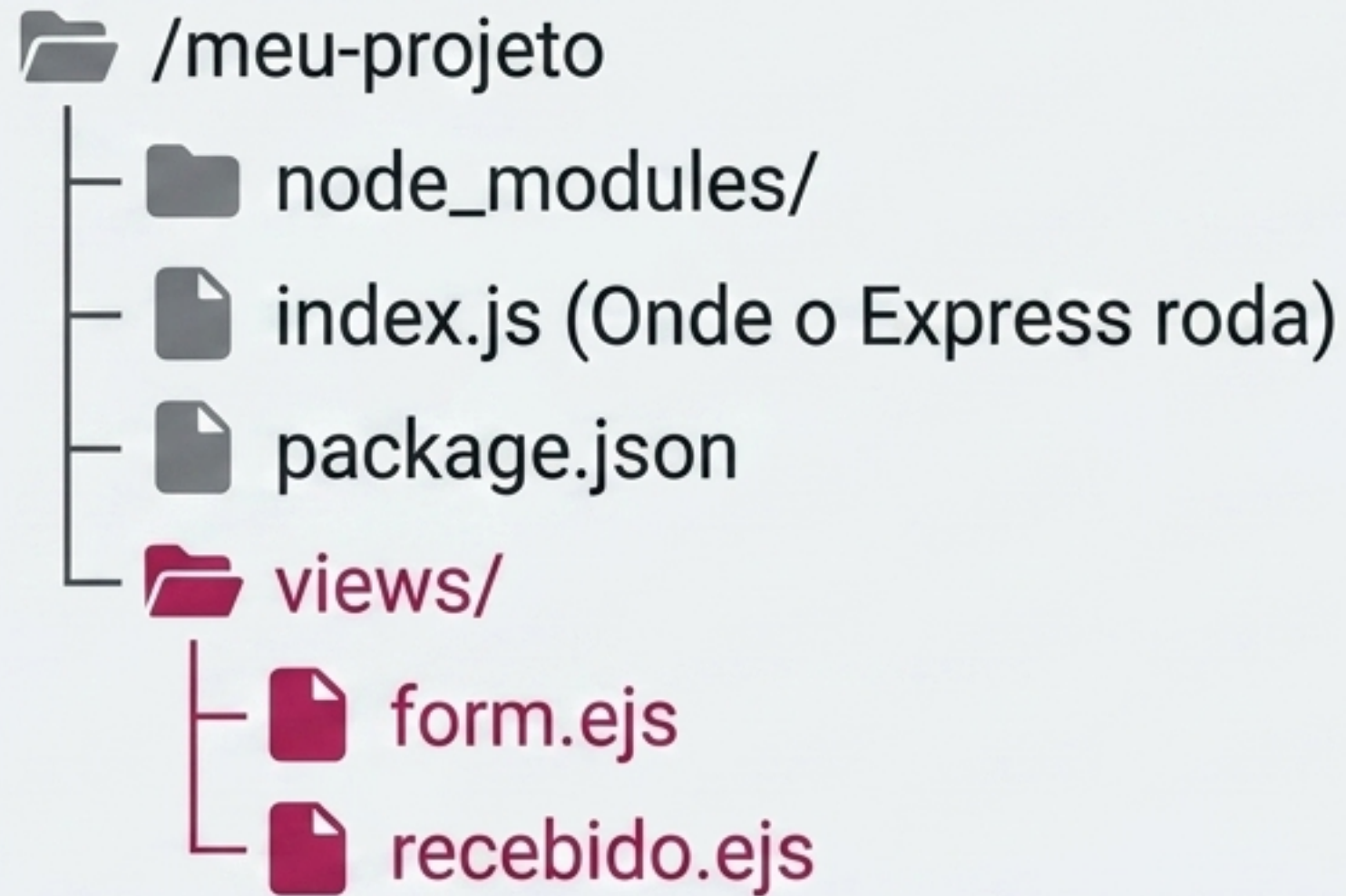
```
const express = require('express');  
const app = express();
```

```
// Define o EJS como o motor de  
visualização
```

```
app.set('view engine', 'ejs');
```

```
app.listen(8080, () =>  
  console.log('Servidor on-line!'));
```


A Estrutura de Diretórios



A Regra de Ouro

- Por padrão, o Express sempre procura os templates dentro de uma pasta chamada exatamente de 'views' na raiz do projeto.
- Os arquivos HTML dentro da pasta recebem a extensão '.ejs' (ex: 'index.ejs' em vez de 'index.html').

Dicionário de Tags EJS

Como o motor diferencia HTML estático de código JavaScript dinâmico.

Tag	Função	Resultado Visual
<code><% ... %></code> (Scriptlet)	Fluxo de controle (if, for, while). Variáveis declaradas aqui.	Sem saída. O código executa, mas nada é impresso no HTML.
<code><%= ... %></code> (Output Escapado)	Imprime o valor de uma variável no template.	HTML Seguro. Converte caracteres especiais em strings (evita ataques XSS).
<code><%- ... %></code> (Output Raw)	Renderiza o valor exatamente como ele é.	HTML Renderizado. Útil se a variável contém tags HTML (ex: <code><h1></code>) que precisam ser formatadas.

Injetando Dados: Do Express para o HTML

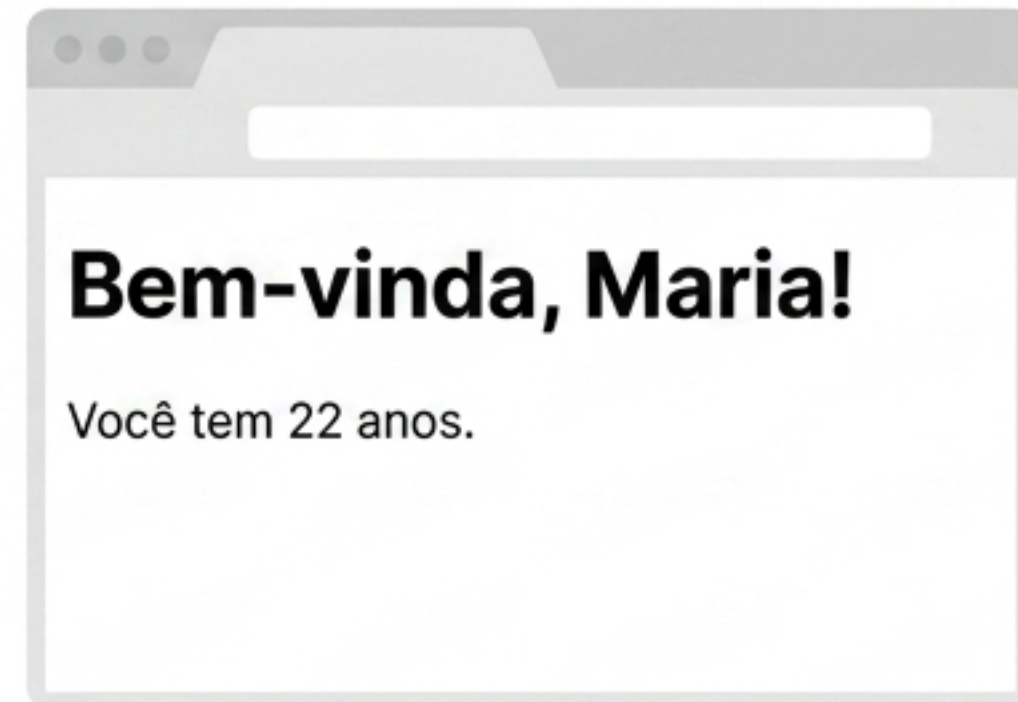
Controller (index.js)

```
app.get('/perfil', (req, res)
=> {
  const dados = { nome:
'Maria', idade: 22 };
  // Renderiza 'perfil.ejs'
enviando 'dados'
  res.render('perfil', dados);
});
```

View (views/perfil.ejs)

```
<html>
  <body>
    <h1>Bem-vinda, <%= nome
%> </span>!</h1>
    <p>Você tem <%= idade %>
</span> anos.</p>
  </body>
</html>
```

Saída Final (Navegador)



Lógica de Programação dentro do Template

Condicionais (if/else)

```
<% if (idade >= 18) { %>
  <p>Usuário Maior de Idade.</p>
<% } else { %>
  <p>Usuário Menor de Idade.</p>
<% } %>
```

A tag `<%` abre o bloco JS, e é necessário usar outra tag `<% }` para fechar as chaves do if e do else.

Repetição (Loops - for)

```
<ul>
  <% for(let i = 0; i < frutas.length; i++) { %>
    <li> <%= frutas[i] %> </li>
  <% } %>
</ul>
```

O HTML estático `<li>` é repetido dinamicamente para cada item do array gerando uma lista perfeita.

Fluxo Completo: Processando Formulários (POST)



Nome

Sobrenome

1. O Formulário (form.ejs)

```
<form action='/cadastro' method='POST'>...
```



2. O Middleware & Rota (index.js)

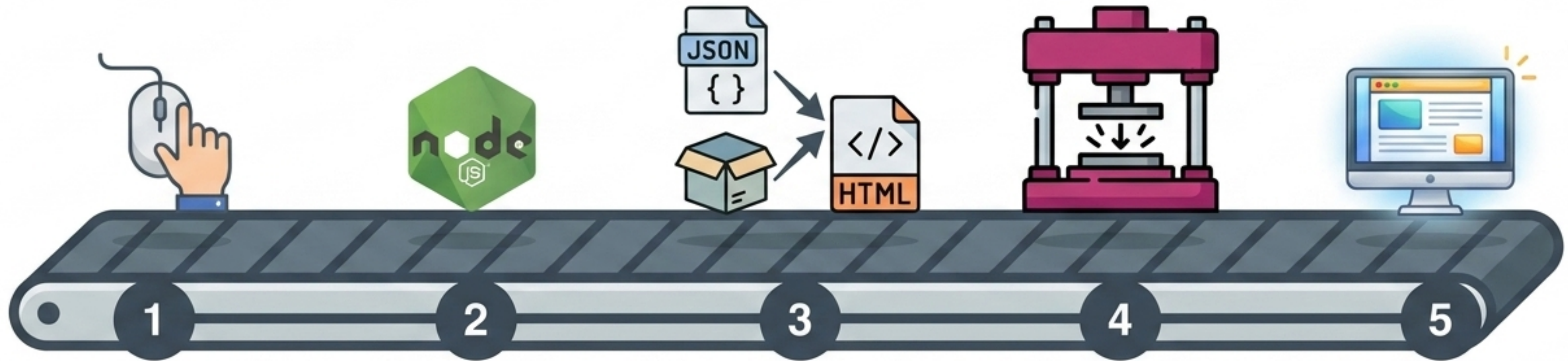
```
app.use(express.urlencoded({extended: true}));  
app.post('/cadastro', (req, res) => {  
  const nome = req.body.nome;  
  const sobrenome = req.body.sobrenome;  
  res.render('recebido', { nomePagina: nome, sobrenomePagina: sobrenome });  
});
```



3. O Resultado (views/recebido.ejs)

```
<h1>Cadastro Confirmado, <%= nomePagina %> <%= sobrenomePagina %>!</h1>
```


Síntese: O Ciclo de Vida da Renderização



1. Request:

Cliente acessa a **rota /perfil**.

2. Controller:

Express recebe requisição e busca dados. Aciona **res.render()**.

3. Raw Materials:

Junta o **HTML Estático (.ejs)** com o **JSON Dinâmico**.

4. The Factory:

EJS compila, executa a lógica e **substitui variáveis**.

5. Response:

Servidor devolve **HTML puro**. O cliente nunca vê o código EJS.

Referências Bibliográficas

- DUARTE, Luiz. Programação Web com Node.js: Completo, do Front-end ao Back-end. LuizTools. Edição do Kindle.
- EXPRESS.JS. Documentação Oficial. Disponível em: <http://expressjs.com/pt-br/>
- EJS. Embedded JavaScript templating. Disponível em: <https://ejs.co/#docs>
- FELTRIN, Fernando. JavaScript do ZERO à Programação Orientada a Objetos. 2023.
- MDN WEB DOCS. JavaScript Docs. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>
- NODE.JS. Documentação Oficial. Disponível em: <https://nodejs.org/en/docs/>